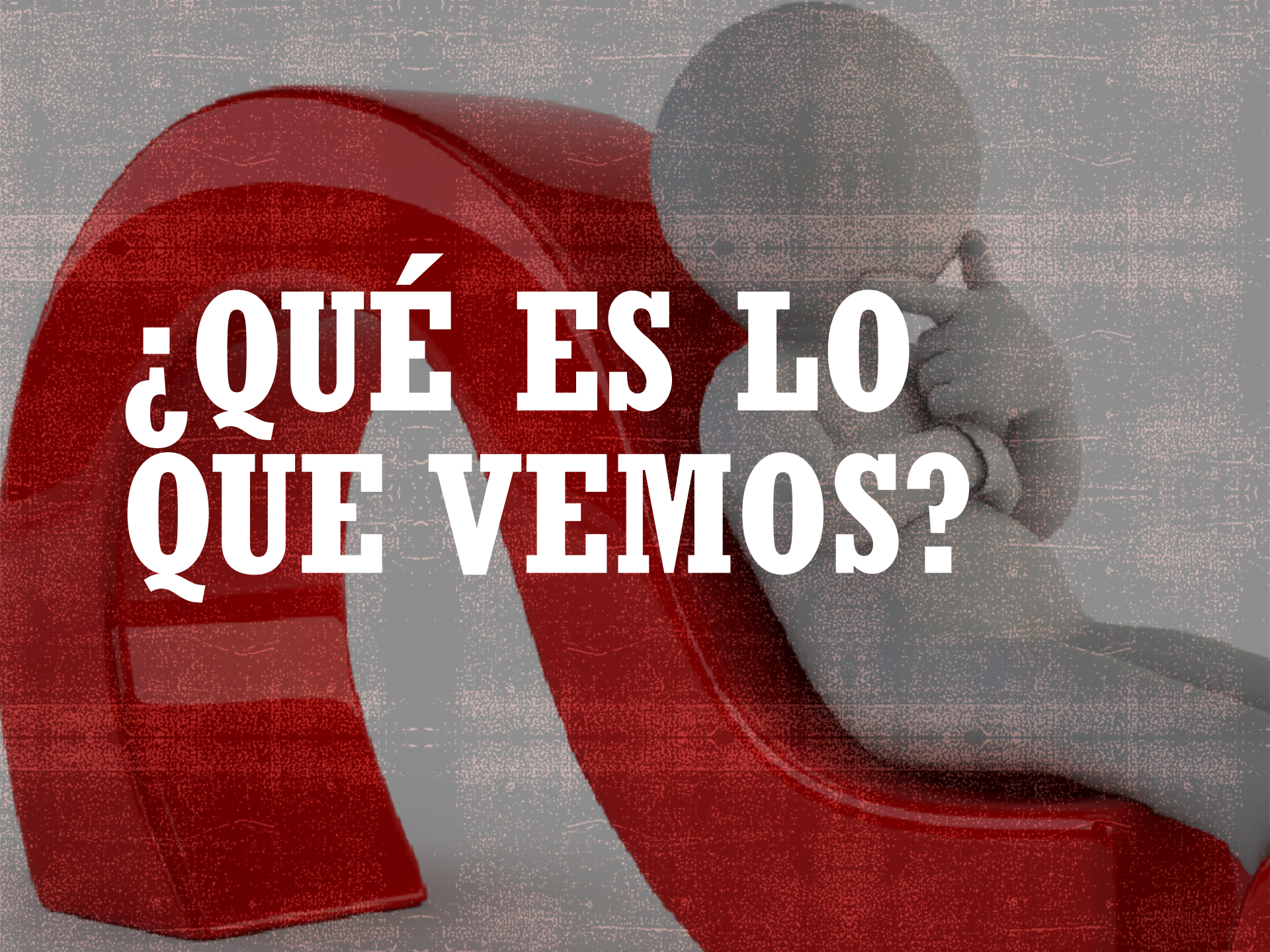


ENTORNOS DE DESARROLLO

UD 1: DESARROLLO DE SOFTWARE

A person wearing a red protective suit and a grey mask is shown from the chest up. Their right hand is raised towards their face, with fingers slightly curled. The background is a plain, light-colored wall.

**¿QUÉ ES LO
QUE VEMOS?**



MATRIX



JUEGOS DE GUERRA



HACKERS



MR. ROBOT



YOUTUBE (IBAI)

```
root@kali: ~
File Edit View Search Terminal Help
0123456789012345
o 8,1
You win
0123456789012345
+-----+
0|000000111110000|0
1|000000111110000|1
2|0000000112110000|2
3|000000011000000|3
4|000000011000000|4
5|0111
6|0111
7|1232
8|1121 xrwPKDXScCrybsY9mY06MibtbbZ9NH5eHzNqy/X2wpbWzSPRwmIjEG2qw0V0sNX4zYk26Ec5pAcs0hVV
9|2333 k2nUVJXPGUXq2BgNbDg/MmRDxmB1 fAHqj6TiKubPRNBCLGQY1xdXz==
0|1212 Unable to load savegame (checksum)
1|1213 0123456789012345
2|0001 101221001112
3|0113 013111001113
4|0112 011131001114
5|0112 012210000015
+-----+
0123456789012345
root@kali: ~
File Edit View Search Terminal Help
1|1213 0123456789012345
2|0001 101221001112
3|0113 013111001113
4|0112 011131001114
5|0112 012210000015
+-----+
0123456789012345
root@kali: ~
File Edit View Search Terminal Help
import base64
import pickle
import hashlib
from minesweeper import Field as F
# greetz to 29c3ctf osiris blue lotus
```

```
minesweeper.py + (~) - VIM
File Edit View Search Terminal Help
self.open(y-1, x+1)
self.open(y, x-1)
self.open(y, x+1)
self.open(y+1, x-1)
self.open(y+1, x)
self.open(y+1, x+1)
return (True, None)

def flag(self, y, x):
    point = (int(y), int(x))
    if point[0] < 0 or point[0] >= self.h:
        return "Illegal point"
    if point[1] < 0 or point[1] >= self.w:
        return "Illegal point"
    if point in self.opened:
        return "Already opened"
    if point in self.flagged:
        self.flagged.remove(point)
    else:
        bisect.insort(self.flagged, point)
    return None

def load(self, data):
    self.dict = pickle.loads(data)

def save(self):
    return pickle.dumps(self.__dict__, 1)

def write(self, stream):
    mine = 0
    open = 0
    flag = 0
    screen = " " + ("0123456789" * ((self.w + 9) / 10))[0:self.w] + "\n + " + (
        for y in xrange(0, self.h):
            have_mines = mine < len(self.mines) and self.mines[mine][0] == y
            have_opened = open < len(self.opened) and self.opened[open][0] == y
            have_flagged = flag < len(self.flagged) and self.flagged[flag][0] == y
            screen += chr(0x30 | (y % 10)) + "|"
            for x in xrange(0, self.w):
```

MR. ROBOT



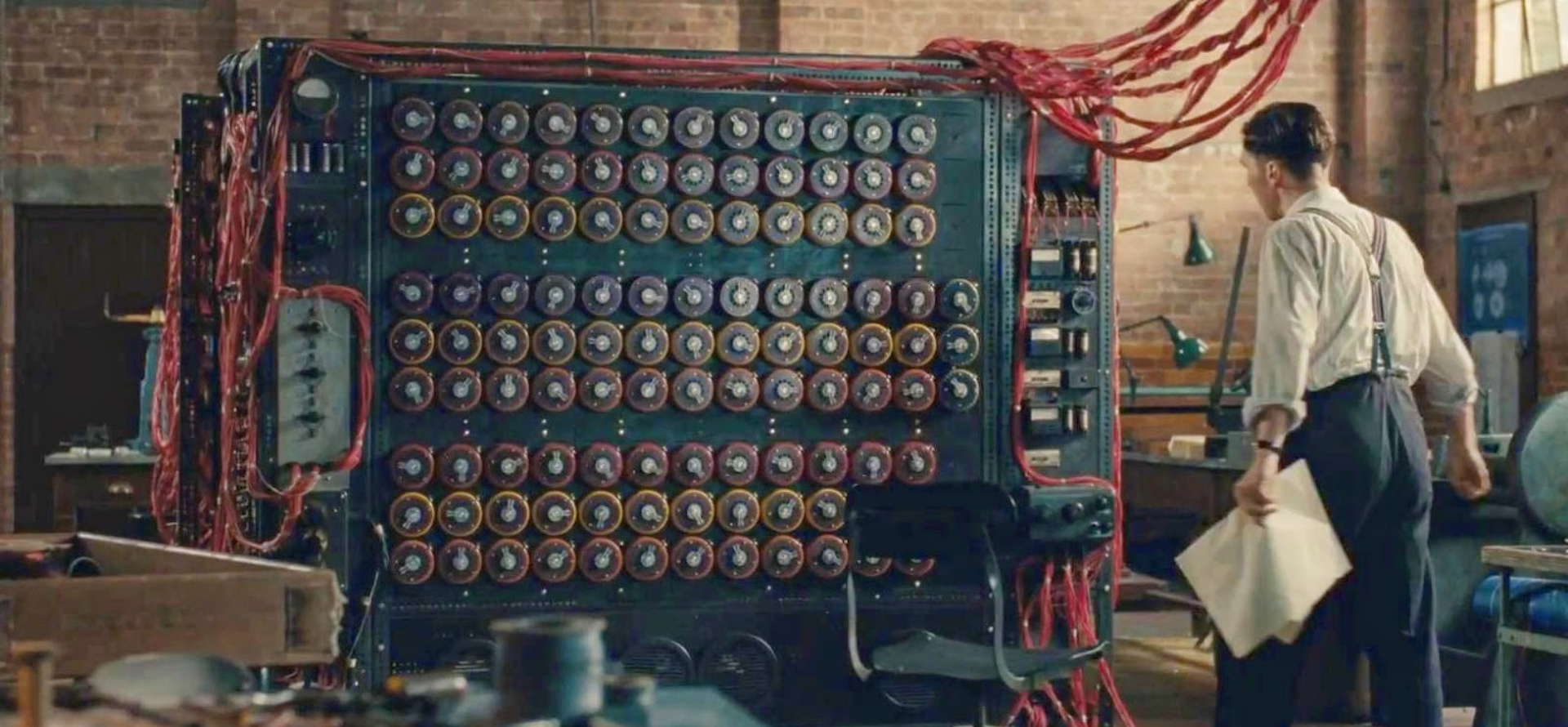
LA PATRULLA CANINA



PIRATAS DE SILICON VALLEY



LOS SIMPSON



DESCIFRANDO ENIGMA

ÍNDICE

- Programa informático
- Lenguajes de programación
- Obtención de código ejecutable
- Proceso de desarrollo
- Roles de desarrollo



1.1 PROGRAMA INFORMÁTICO

- **Instrucción:** “Conjunto de símbolos que representa una orden para el ordenador”
- **Programa informático:** “Conjunto de instrucciones que se dan a un ordenador indicándole las tareas que se desea que realice”:
 - Las instrucciones están escritas en un determinado lenguaje de programación.
 - Ese lenguaje de programación es entendible por el programador.
 - Se necesita de un elemento intermedio (compilador), para que lo traduzca a un “lenguaje/código máquina”.



1.1 PROGRAMA INFORMÁTICO (CONT)

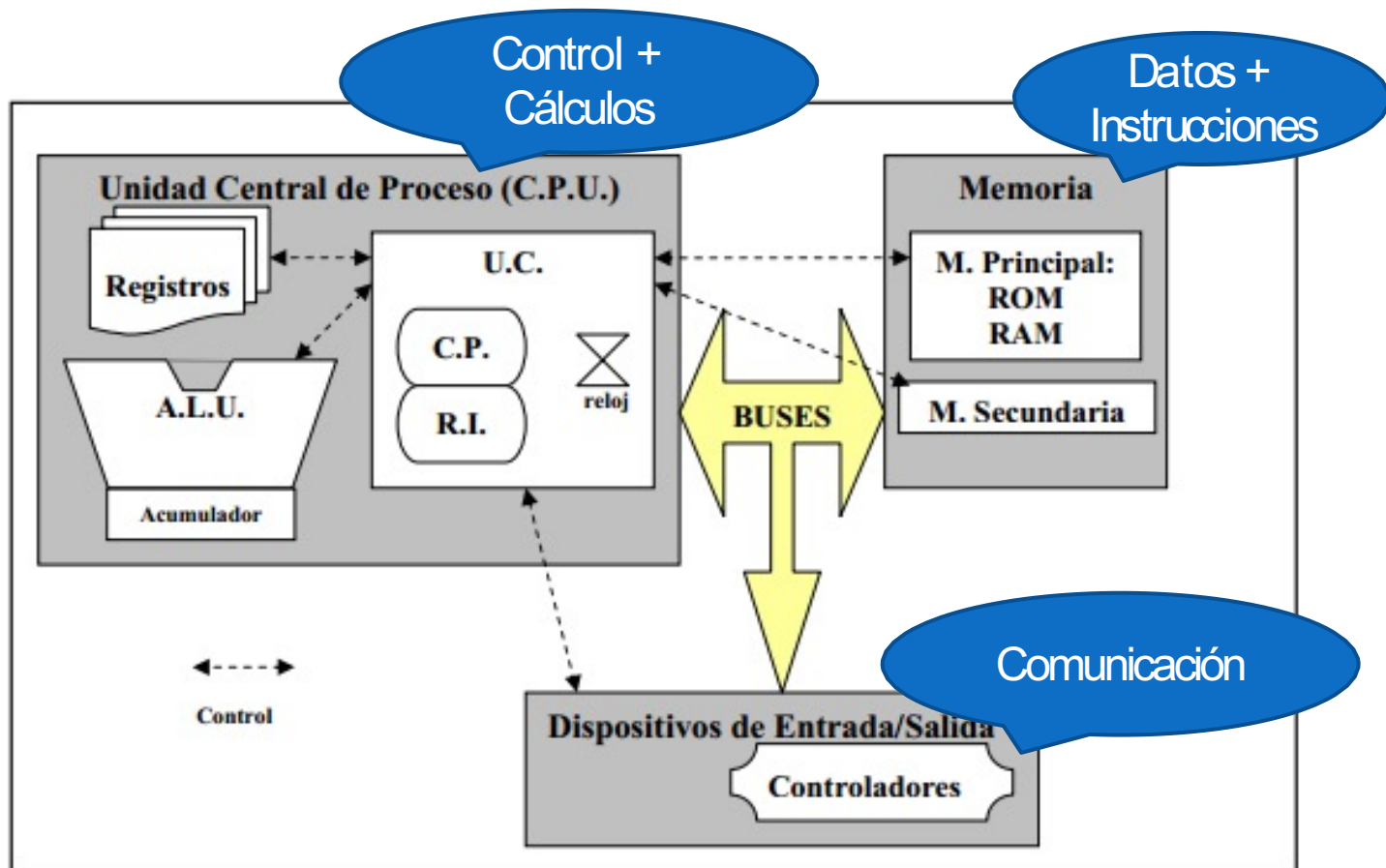
- Software: “Conjunto de programas escritos para un ordenador”:
- Software de sistema o base: Controla y gestiona el hardware y el software de aplicación. Ejemplo: Sistema Operativo (SO)
- Software de aplicación: Se ejecuta en el SO y es traducido a un lenguaje máquina que entiende el SO.

C1.1: Pensar en posibles ejemplos para cada uno de los tipos de software.



1.1.1 MÁQUINA DE VON NEUMANN

- 1945: Concepto de “programa almacenado”.



1.2 LENGUAJES DE PROGRAMACIÓN

- Definición: “Es un lenguaje formal y artificial diseñado para ser entendible por humanos y expresar procesos que pueden ser llevados a cabo por máquinas.”



1.2.1 CLASIFICACIÓN Y CARACTERÍSTICAS

- A) Nivel de abstracción:
 - L. de bajo nivel:
 - Primera generación: código máquina.
 - L. de medio nivel:
 - Segunda generación: lenguaje ensamblador.

```
DOSSEG
.model small
.stack 100h
.data
msgHello DB "Hola mundo!",13,10,"$"
.code
    mov ax,@data           ; Strichpunkt leitet Kommentar ein!
    mov ds,ax              ; Datensegment initialisiert

    mov dx,offset msgHello ;
    mov ah,9               ; Ausgabe eines String
    int 21h                ;

    mov ax,4C00h           ; Programm beenden
    int 21h                ;
END
```



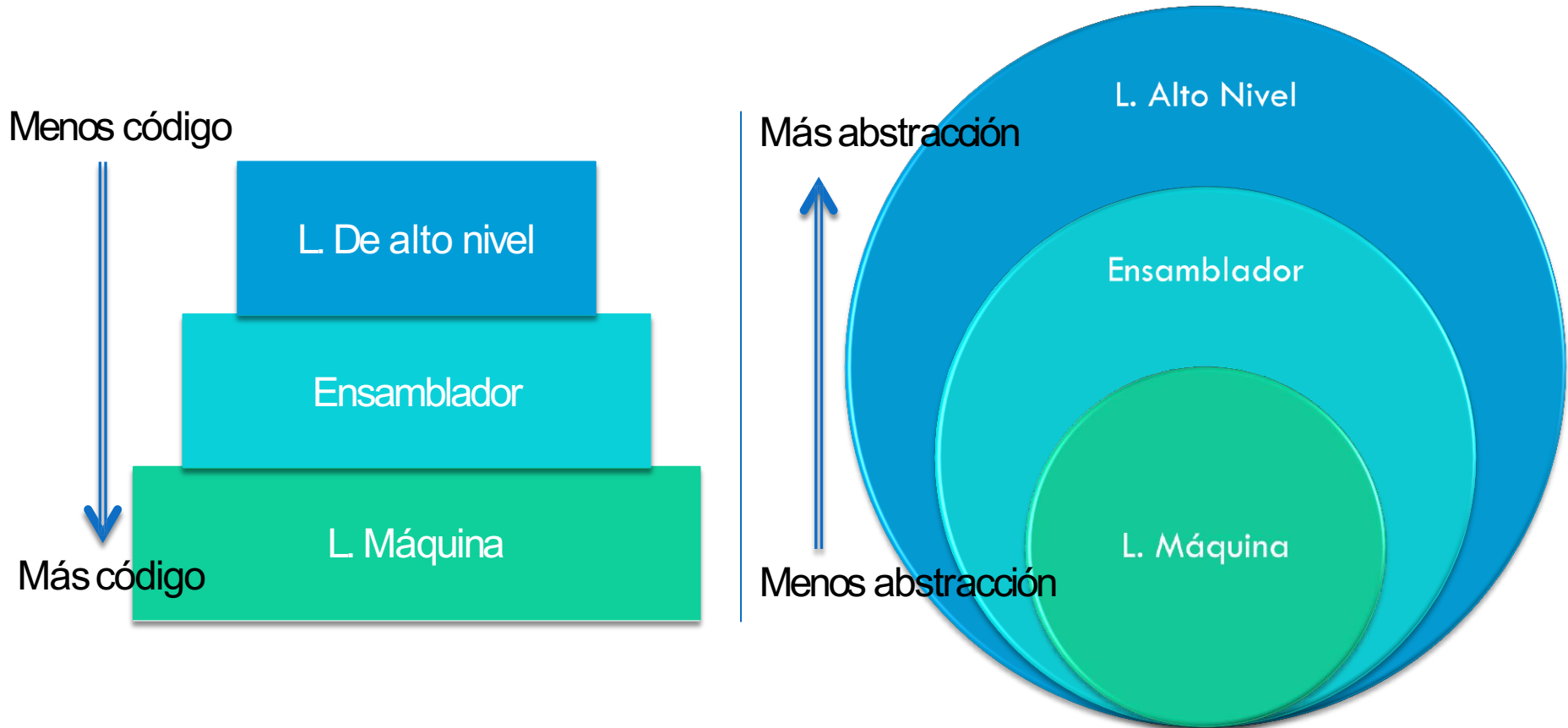
1.2.1 CLASIFICACIÓN Y CARACTERÍSTICAS

- L. de alto nivel:
 - Tercera generación (desde década de los 60): Se separan del lenguaje máquina. Ejemplos: Matlab, C#, Java...
 - Cuarta generación: Permiten crear aplicaciones combinando piezas “prefabricadas” y que además se acercan más al lenguaje humano. Ejemplo: VisualBasic.
 - Quinta generación: Se asocia a lenguajes utilizados para la inteligencia artificial.



1.2.1 CLASIFICACIÓN Y CARACTERÍSTICAS

➤ Dos formas de verlo:



1.2.1 CLASIFICACIÓN Y CARACTERÍSTICAS

- B) Forma de ejecución:
 - L. interpretados: Aquellos que ejecutan las instrucciones directamente.
 - L. compilados: Estos lenguajes se traducen/compilan a un código objeto. Pudiendo ser éste código binario.
 - L. virtuales: Es un subtipo del anterior, el cual genera un código intermedio *bytecode*, el cual es ejecutado en una máquina virtual. Portabilidad/Multisistema.



1.2.1 CLASIFICACIÓN Y CARACTERÍSTICAS

- C) Paradigma de programación: Estilo de desarrollo empleado. Los lenguajes de alto nivel suelen soportar múltiples paradigmas.
 - **P. imperativo:** La programación se considera una secuencia de instrucciones a realizar a través de una o más funciones (se detalla el algoritmo para resolver el problema).
 - **P. procedimental/estructurada:** El lenguaje se divide en partes más pequeñas: procedimientos y funciones. Ejemplo: C.
 - **P. orientado a objetos:** Hace uso de estructuras de datos llamadas objetos que aglutinan propiedades y métodos conjuntamente con sus interacciones.
 - **P. declarativo:** Se detalla la solución del problema en lugar de decir cómo llegar a la solución. La solución es hallada internamente mediante mecanismos de inferencia.
 - **P. funcional:** Se evita declarar variables, usando funciones de forma recursiva. Ejemplo: Haskell.
 - **P. lógico:** Se definen reglas lógicas para ser interpretadas mediante inferencias lógicas. Ejemplo: Prolog



ACTIVIDADES

UD1_A1. Buscar lenguajes de programación más utilizados y clasificarlos según el nivel, forma de ejecución y paradigma.

Pista: puedes utilizar el índice TIBO, survey stackoverflow, ...



1.3 OBTENCIÓN DE CÓDIGO EJECUTABLE

➤ Tipos de códigos:

A. Código fuente:

- ✓ Conjunto de sentencias entendibles por el programador que componen el programa o una parte de ello.
- ✓ Suele estar almacenado en un fichero del tipo texto como los que se pueden abrir por ejemplo, con el bloc de notas o Wordpad en los entornos Windows.
- ✓ El código fuente estará escrito en un lenguaje de programación determinado, elegido por el programador, (Basic, C, C++, C#, Java, Perl, Python, PHP)



1.3 OBTENCIÓN DE CÓDIGO EJECUTABLE

➤ Tipos de códigos :

B. Código objeto:

- ✓ Conjunto de instrucciones y datos escritos en un lenguaje que entiende el ordenador directamente: binario o código máquina.
- ✓ Proviene de la traducción/compilación del código fuente.
 - ✓ En caso de tratarse de un lenguaje de programación compilado → código máquina.
 - ✓ Si es un lenguaje de programación virtual → *bytecode*.
- ✓ Es un fragmento del programa final y es específico de la plataforma de ejecución.



1.3 OBTENCIÓN DE CÓDIGO EJECUTABLE

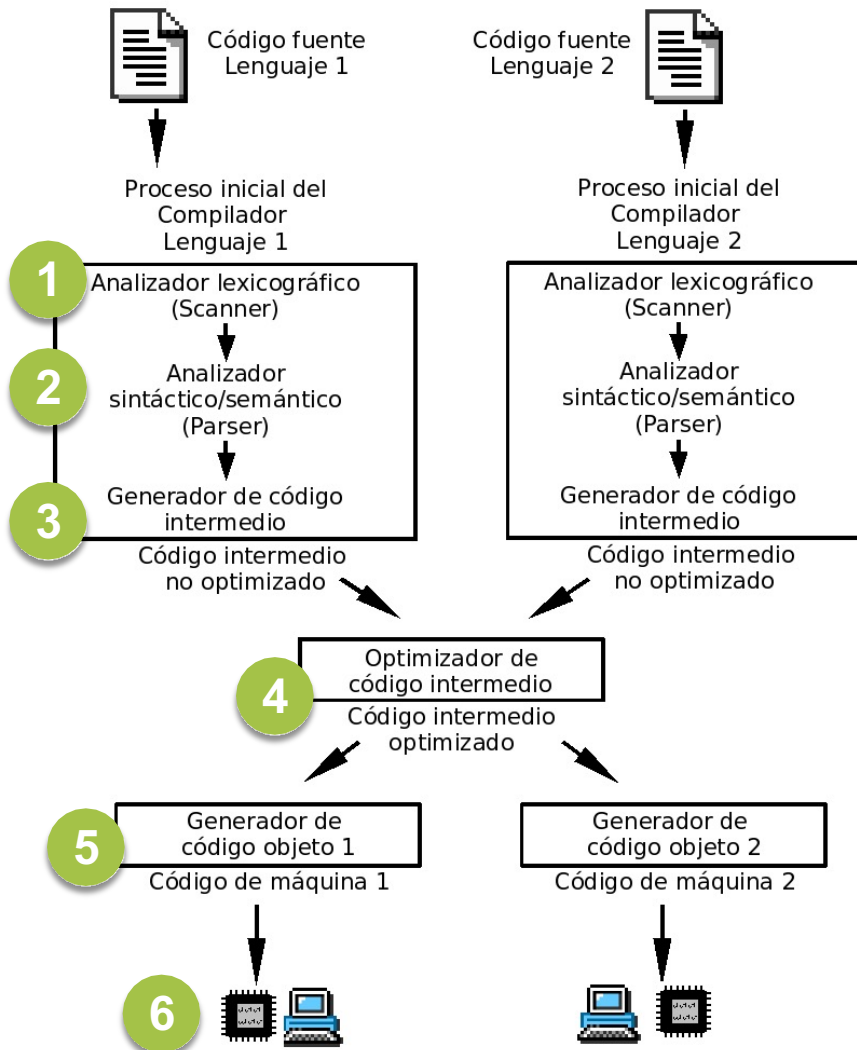
➤ Tipos de códigos:

c. Código ejecutable:

- ✓ Reúne el código objeto generado por los programadores junto con las librerías propias del entorno (enlazador/interprete) componiendo el programa final.
- ✓ Este es el código que ejecutan los usuarios del sistema, y es específico para una plataforma concreta. Ejemplo: *.exe para Windows.



1.3.2 PROCESO DE COMPILACIÓN

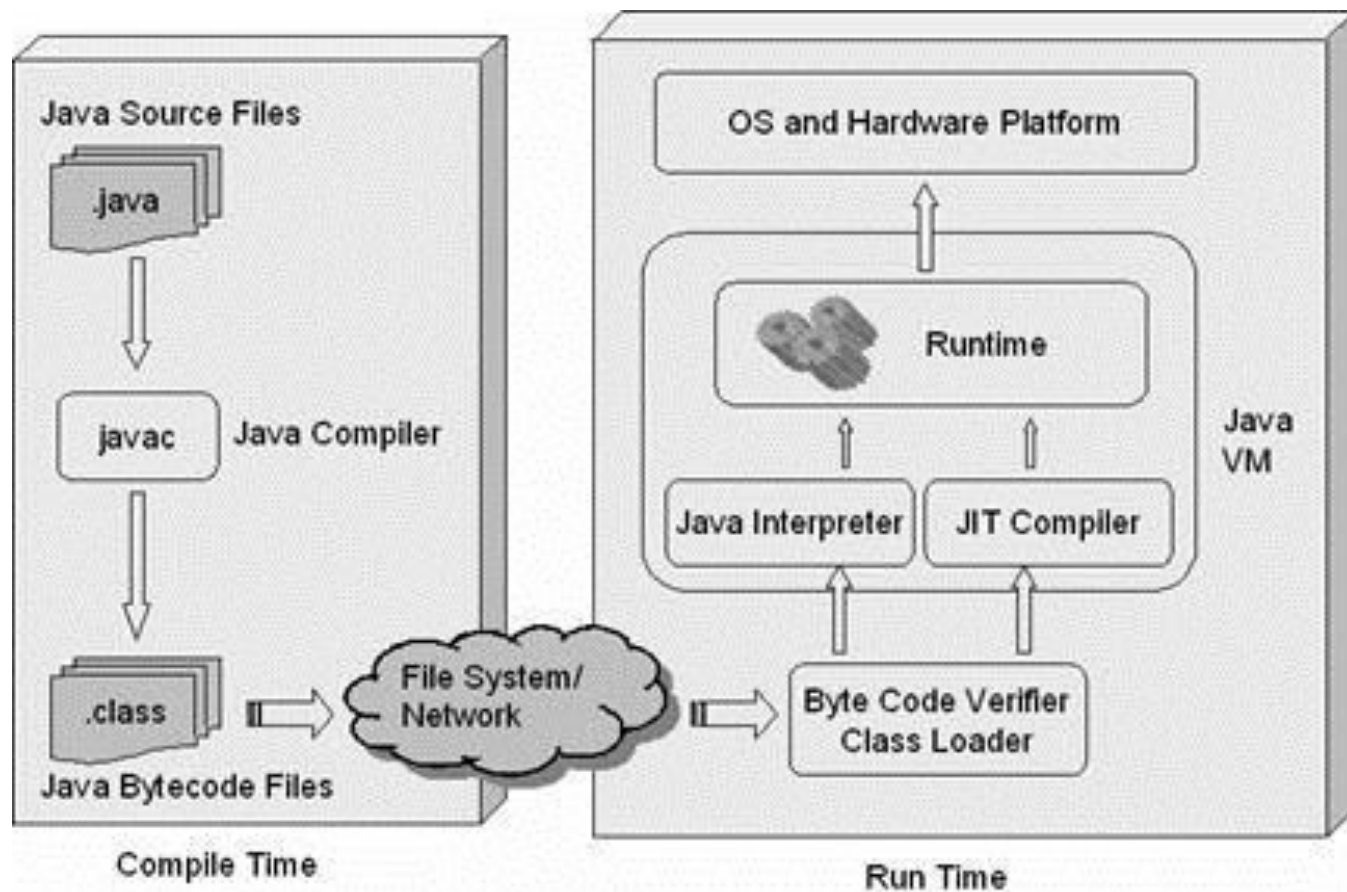


1. Análisis lexico
2. Análisis sintáctico/semántico
3. Generador de cód. intermedio
4. Optimización de cód. intermedio
5. Generación de código objeto
6. Enlazador de librerías (pág. Siguiende)



1.3.2 PROCESO DE COMPILACIÓN

➤ Pasos de compilación en Java:



ACTIVIDADES

UD1_A2. Compilación y ejecución de un programa Java. Documentar los pasos.

AMPLIACIÓN

UD1_A3. Compilación de un programa C++ utilizando diferentes modos de optimización.

INVESTIGACIÓN

AD1_A4. Decompilación de un programa Java. Pista: Cavj Java Decompiler

AD1_A5. Ofuscación de código.

